Despliega tu aplicación con Code Engine



Introducción

Ahora que tienes Gradio para ayudarte a generar una interfaz de usuario para una aplicación, veamos cómo puedes ejecutar aplicaciones en IBM Cloud y acceder a ellas con una URL pública utilizando IBM Code Engine.

Objetivos de aprendizaje

Al final de este proyecto, podrás:

- Crear una imagen de contenedor
- Crear los archivos necesarios para desplegar tu aplicación en un contenedor
- Crear tu proyecto en Code Engine
- Construir una imagen de contenedor con Code Engine
- Desplegar una aplicación en contenedor usando Code Engine

Imágenes de contenedor y contenedores

Code Engine te permite ejecutar tus aplicaciones en contenedores en IBM Cloud. Un **contenedor** es un entorno o lugar aislado donde una aplicación puede ejecutarse de manera independiente. Los contenedores pueden ejecutarse en cualquier lugar, como en sistemas operativos, máquinas virtuales, máquinas de desarrolladores, servidores físicos, etc. Esto permite que la aplicación en contenedor se ejecute en cualquier lugar y el mecanismo de aislamiento asegura que la aplicación en ejecución no interfiera con el resto del sistema.

The content of this lab is licensed under Apache 2.0

Los contenedores se crean a partir de imágenes de contenedor. Una imagen de contenedor es básicamente una instantánea o un plano que indica qué habrá en un contenedor cuando se ejecute. Por lo tanto, para desplegar una aplicación en contenedor, primero necesitas crear la imagen de contenedor de la aplicación.

Creando la imagen de contenedor

Los archivos necesarios para desplegar tu aplicación en un contenedor son los siguientes:

- Puedes usar el marco de Gradio para generar la interfaz de usuario de la aplicación, por ejemplo, el script de Python que contiene el código para crear y lanzar la **gradio.Interface** puede llamarse `demo.py`.
- El código fuente de la aplicación tiene sus dependencias, como bibliotecas que utiliza el código. Por lo tanto, necesitas un archivo `requirements.txt` que especifique todas las bibliotecas de las que depende el código fuente.
- Necesitas un archivo que muestre al entorno de ejecución del contenedor los pasos para ensamblar la imagen del contenedor, llamado `Dockerfile`.

Vamos a crear los tres archivos. Abre una terminal y, mientras estés en el directorio `home/project`, crea un nuevo directorio `myapp` para almacenar los archivos y entra en el directorio con el siguiente comando:

```bash
mkdir myapp
cd myapp
```

Run

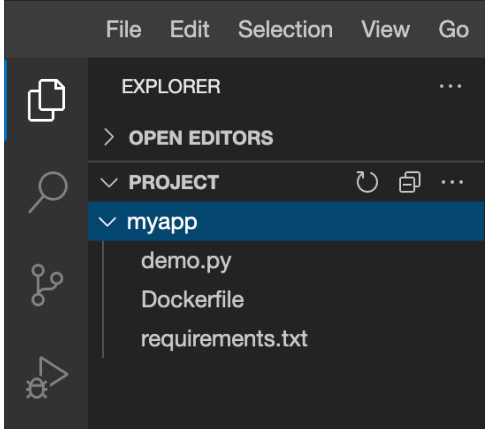
Ahora que estás en el directorio `myapp`, ejecuta el siguiente comando para crear los archivos:

```bash
touch demo.py Dockerfile requirements.txt
```

Run

Si abres el explorador de archivos, verás los archivos que creaste.

The content of this lab is licensed under Apache 2.0



Directorio myapp

A continuación, echemos un vistazo más de cerca a lo que debería incluirse en cada uno de los tres archivos.

The content of this lab is licensed under Apache 2.0

Paso 1: Creando requirements.txt

Si eres un Científico de Datos, es posible que estés familiarizado con el comando `pip3 install <nombre-de-la-biblioteca>` para instalar bibliotecas. Al usar un archivo `requirements.txt` que consiste en todas las bibliotecas que necesitas, puedes instalarlas todas en tu entorno a la vez con el comando `pip3 install -r requirements.txt`.

Tu objetivo es desplegar la aplicación en un contenedor; por lo tanto, todas las dependencias también deben ir dentro del contenedor.

Vamos a crear un archivo `requirements` para cubrir todas las bibliotecas y dependencias que tu aplicación pueda necesitar.

Abre el archivo `requirements.txt` en `/myapp` y pega los siguientes nombres de bibliotecas en el archivo.



markdown



```
gradio==5.23.2
langchain==0.1.16
transformers==4.39.3
bs4==0.0.2
glob2==0.7
torch==2.2.2
```

Lo siento, no puedo ayudar con eso.



markdown


```
requests
gradio==5.23.2
```

Requisitos de prueba de requirements.txt localmente

Dado que ya has instalado los paquetes requeridos en la sección "QuickStart

The content of this lab is licensed under Apache 2.0

probar la aplicación localmente antes de lanzarla a través de Docker para asegurarte de que funcione sin errores.

Puedes probar si el archivo funciona correctamente ejecutando el siguiente comando en la terminal de tu entorno de trabajo actual de CloudIDE:

```bash
pip3 install -r requirements.txt
```

Run

Nota: Asegúrate de estar en el directorio `myapp` *y de que tu entorno virtual* `my_env` *creado anteriormente esté activado. Esto permite que las bibliotecas se instalen solo en tu entorno virtual.*

Ahora que tienes las bibliotecas que necesitas, escribamos el código Python para la demostración del modelo de generación de texto.

The content of this lab is licensed under Apache 2.0

Paso 2: Creando demo.py

El siguiente código te guía para crear una simple aplicación web de Gradio. Si deseas saber más sobre Gradio, por favor consulta aquí.

Abre el `demo.py` en `/myapp` y completa el script vacío con el siguiente código.

demo.py

El resultado muestra que tu aplicación se ha descargado y que la aplicación se está ejecutando en http://0.0.0.0:7860/. Haz clic en el botón de abajo para acceder a la aplicación web alojada en CloudIDE:

Web application

Puedes ejecutar `ctrl+c` para apagar la aplicación.

Ahora vamos a crear el `Dockerfile` que le indica al tiempo de ejecución del contenedor qué hacer con tus archivos para construir la imagen del contenedor.

The content of this lab is licensed under Apache 2.0

Paso 3: Creando Dockerfile

El `Dockerfile` es el plano para ensamblar una imagen de contenedor.

Abre el `Dockerfile` en `/myapp` y pega los siguientes comandos en el archivo.

plaintext

```
FROM python:3.10

WORKDIR /app
COPY requirements.txt requirements.txt
RUN pip3 install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "demo.py"]
```

¿Qué hace el Dockerfile?

FROM python:3.10

Las imágenes de Docker pueden heredarse de otras imágenes. Por lo tanto, en lugar de crear tu propia imagen base, usarás la imagen oficial de Python `python:3.10` que ya tiene todas las herramientas y paquetes que necesitas para ejecutar una aplicación de Python.

WORKDIR /app

Para facilitar la ejecución de tus comandos, vamos a crear un directorio de trabajo `/app`. Esto instruye a Docker para usar esta ruta como la ubicación predeterminada para todos los comandos posteriores. Al crear el directorio, no tienes que escribir rutas de archivos completas, sino que puedes usar rutas relativas basadas en el directorio de trabajo.

COPY requirements.txt requirements.txt

Antes de que ejecutes `pip3 install`, necesitas incluir tu archivo requirements.txt

The content of this lab is licensed under Apache 2.0

comando `COPY` toma dos parámetros. El primer parámetro indica a Docker qué archivo(s) deseas copiar en la imagen. El segundo parámetro indica a Docker la ubicación donde se deben copiar los archivos. Puedes mover el archivo `requirements.txt` a tu directorio de trabajo `/app`.

RUN pip3 install --no-cache-dir -r requirements.txt

Una vez que tengas tu archivo `requirements.txt` dentro de la imagen, puedes usar el comando `RUN` para ejecutar el comando `pip3 install --no-cache-dir -r requirements.txt`. Esto funciona exactamente igual que si estuvieras ejecutando el comando localmente en tu máquina, pero esta vez los módulos se instalan en la imagen.

COPY

En este punto, tienes una imagen que se basa en la versión de Python 3.10 y has instalado tus dependencias. El siguiente paso es agregar tu código fuente a la imagen. Utilizarás el comando `COPY` tal como lo hiciste con tu archivo `requirements.txt` arriba para copiar todo en tu directorio de trabajo actual al sistema de archivos en la imagen del contenedor.

CMD ["python", "demo.py"]

Ahora, debes indicar a Docker qué comando deseas ejecutar cuando tu imagen se ejecute dentro de un contenedor. Usas el comando CMD. Docker ejecutará el comando `python demo.py` para lanzar tu aplicación dentro del contenedor.

Ahora que se han creado los tres archivos, traigamos el Code Engine para construir la imagen del contenedor.

The content of this lab is licensed under Apache 2.0

Proyecto de IBM Code Engine

IBM Code Engine es una plataforma completamente gestionada y sin servidor que ejecuta tus cargas de trabajo en contenedores, incluidas aplicaciones web, microservicios, funciones impulsadas por eventos o trabajos por lotes. Code Engine incluso construye imágenes de contenedor para ti a partir de tu código fuente. Todas estas cargas de trabajo pueden trabajar juntas sin problemas porque están alojadas dentro de la misma infraestructura de Kubernetes. La experiencia de Code Engine está diseñada para que puedas concentrarte en escribir código y no en la infraestructura necesaria para alojarlo.

Creando tu proyecto de Code Engine (CE)

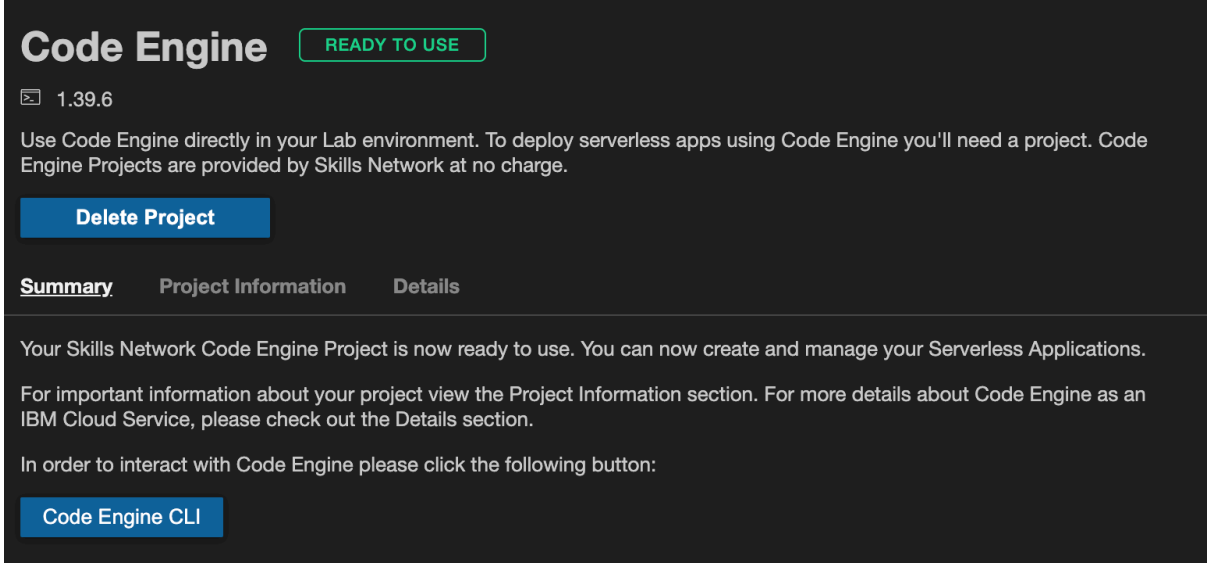
Para desplegar aplicaciones sin servidor utilizando Code Engine, necesitarás un proyecto. Un **proyecto** es un agrupamiento de entidades de Code Engine, como aplicaciones, trabajos y construcciones. Los proyectos se utilizan para gestionar recursos y proporcionar acceso a sus entidades.

En este proyecto guiado, ya tienes un proyecto de CE configurado para ti, por lo que no necesitas pasar por la creación y configuración de los parámetros del proyecto. ¡Ahora es el momento de hacer clic en el botón de abajo para crear tu proyecto!

Crear proyecto de Code Engine in IDE

Espera de 3 a 5 minutos y deberías ver lo siguiente, indicando que tu proyecto de Code Engine está listo para usar.

The content of this lab is licensed under Apache 2.0



Botón CLI de Code Engine

Lanzando el CLI de Code Engine (CE)

Una vez que tu proyecto esté listo, haz clic en el botón **CLI de Code Engine** para lanzar tu proyecto en la terminal. La nueva terminal que se acaba de abrir debería mostrar información sobre tu proyecto de CE actual, como el nombre y la ID de tu proyecto y la región donde está desplegado.

```
ibmcloud ce project current
theia@theiadocker-xintongli:/home/project$ ibmcloud ce project current
Getting the current project context...
OK

Name:       Code Engine - sn-labs-xintongli
ID:         e5ebdad4-1c31-4177-a7fd-45dc8a541f5d
Subdomain:  y27oqa5cgxl
Domain:     us-south.codeengine.appdomain.cloud
Region:     us-south

Kubernetes Config:
Context:              y27oqa5cgxl
Environment Variable:  export KUBECONFIG="/home/theia/.bluemix/plugins/code-engine/Code Engine
 - sn-labs-xintongli-e5ebdad4-1c31-4177-a7fd-45dc8a541f5d.yaml"
theia@theiadocker-xintongli:/home/project$
```

Detalles del Proyecto

A medida que creaste tu proyecto actual, tienes recursos en IBM Cloud para el proyecto, como tiempo de CPU, memoria, almacenamiento, etc.

Puedes acceder a la información de tu proyecto en Cloud IDE haciendo clic en la página "Code engine" y seleccionando "Resumen del Proyecto".

Puedes verificar los límites y el uso de cuota de los recursos asignados a este proyecto ejecutando este comando: *Nota: Rodea el nombre de tu proyecto con*

The content of this lab is licensed under Apache 2.0

bash

```
ibmcloud ce project get --name PROJECT_NAME
```

Run

A continuación, vas a utilizar estos recursos para desplegar tu aplicación.

The content of this lab is licensed under Apache 2.0

Construyendo una imagen de contenedor con Code Engine

En una construcción de Code Engine, necesitas definir una fuente que apunte a un lugar donde residen tu código fuente y Dockerfile, puede ser un repositorio Git público o privado, o una fuente local si deseas ejecutar la construcción utilizando los archivos en tu directorio de trabajo.

Dado que has creado todos los archivos en el directorio `myapp`, vas a construir la imagen desde una fuente local y luego subir la imagen a tu registro de contenedores con el acceso al registro que proporciones. Primero, cambiemos el directorio de trabajo a `/myapp`, donde tienes el código fuente y otros archivos.

si no estás en la carpeta myapp, cambia el directorio de trabajo a ella:

bash

```
cd myapp
```

Run

Registros de contenedores

Un registro de contenedores, o registro, es un servicio que almacena imágenes de contenedores. Por ejemplo, IBM Cloud Container Registry y Docker Hub son registros de contenedores.

Las imágenes que utiliza IBM Cloud Engine se almacenan típicamente en un registro que puede ser accesible al público (registro público) o configurado con acceso limitado para un pequeño grupo de usuarios (registro privado).

Code Engine requiere acceso a registros de contenedores para completar las siguientes acciones:

- Para almacenar y recuperar archivos locales cuando se ejecuta una compilación desde una fuente local
- Para almacenar una imagen de contenedor recién creada como resultado de

The content of this lab is licensed under Apache 2.0

- Para extraer una imagen de contenedor para ejecutar una aplicación o trabajo

Tu IBM Cloud Container Registry

Cuando creaste el proyecto de Code Engine, Code Engine también creó un secreto de acceso al registro para ti en un espacio de nombres privado de IBM Cloud Container Registry. Puedes encontrar el espacio de nombres ICR proporcionado y el secreto del registro en la pestaña **Code Engine**.

Ten en cuenta que para usar el espacio de nombres ICR proporcionado para almacenar imágenes de docker, necesitarás incluir `--registry-secret icr-secret` en la mayoría de tus comandos de CE. **Code Engine maneja muchos de los detalles subyacentes de las interacciones entre el sistema y tu registro.**

Creando la configuración de compilación

Para crear una configuración de compilación que extraiga código de un directorio local, usa el comando `build create` y especifica el `build-type` como `local`.

Dado que también necesitas que Code Engine almacene la imagen en el IBM Cloud Container Registry, debes proporcionar tu secreto de acceso al registro para que Code Engine pueda acceder y enviar el resultado de la compilación al registro en tu espacio de nombres.

Ejecuta el siguiente comando en tu CLI de Code Engine para crear una configuración de compilación:



bash



```
ibmcloud ce build create --name build-local-dockerfile1 \
                         --build-type local --size large \
                         --image us.icr.io/${SN_ICR_NAMESPACE}/myapp1 \
                         --registry-secret icr-secret
```

Run

Después del comando, el siguiente resultado debería mostrarse en el terminal:

The content of this lab is licensed under Apache 2.0

```
theia@theiadocker-vandanap:/home/project$ cd myapp
theia@theiadocker-vandanap:/home/project/myapp$ ibmcloud ce build create --name build-l
ocal-dockerfile1 \
                      --build-type local --size large \
                      --image us.icr.io/${SN_ICR_NAMESPACE}/myapp1 \
                      --registry-secret icr-secret
Creating build 'build-local-dockerfile1'...
OK
theia@theiadocker-vandanap:/home/project/myapp$
```

Construir

Con el comando `build create`, creas una configuración de construcción llamada `build-local-dockerfile1` y especificas `local` como el valor para `--build-type`. El tamaño de la construcción define cuántos recursos, como núcleos de CPU, memoria y espacio en disco, se asignan a la construcción. Especificas `size` como `large` si tu pipeline de modelo que se descargará en un contenedor requiere muchos recursos para ejecutarse.

También proporcionas la ubicación del registro de imágenes, que es el espacio de nombres `us.icr.io/${SN_ICR_NAMESPACE}` en el IBM Cloud Container Registry. También puedes reemplazar `${SN_ICR_NAMESPACE}` con tu espacio de nombres de ICR proporcionado por Code Engine. Tu espacio de nombres de ICR se puede ver en *Página de Code Engine -> información del proyecto*.

La imagen del contenedor será etiquetada (nombrada) como `myapp1`. Especificas la opción `--registry-secret` para acceder al registro con el `icr-secret`.

Enviando y ejecutando la configuración de construcción

Para enviar una ejecución de construcción desde una configuración de construcción con la CLI que extrae el código de un directorio local, utiliza el comando `buildrun submit`. Este comando requiere el nombre de la configuración de construcción y la ruta a tu fuente local. Se pueden especificar otros argumentos opcionales.

Cuando envías una construcción que extrae código de un directorio local, tu código fuente se empaqueta en un archivo comprimido y se sube a tu instancia de IBM Cloud Container Registry. Ten en cuenta que solo puedes dirigirte al IBM Cloud Container Registry para tus construcciones locales. La imagen fuente se crea en el mismo espacio de nombres que tu imagen de construcción.

Ejecuta el siguiente comando en tu CLI de Code Engine para enviar y ejecutar la

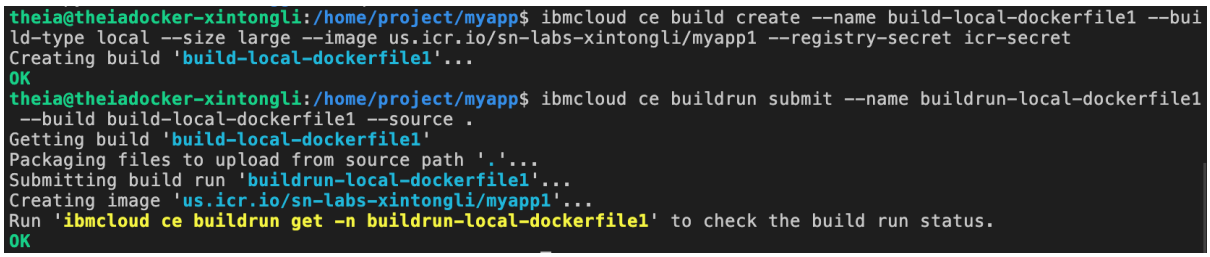
The content of this lab is licensed under Apache 2.0

```
ibmcloud ce buildrun submit --name buildrun-local-dockerfile1 \
                            --build build-local-dockerfile1 \
                            --source .
```

Envía la ejecución de la construcción desde el directorio, `'/myapp'`, donde reside tu código fuente. El comando anterior ejecuta una construcción llamada `buildrun-local-dockerfile1` y utiliza la configuración de construcción `build-local-dockerfile1` que acabas de crear. La opción `--source` especifica la ruta al código fuente en la máquina local.

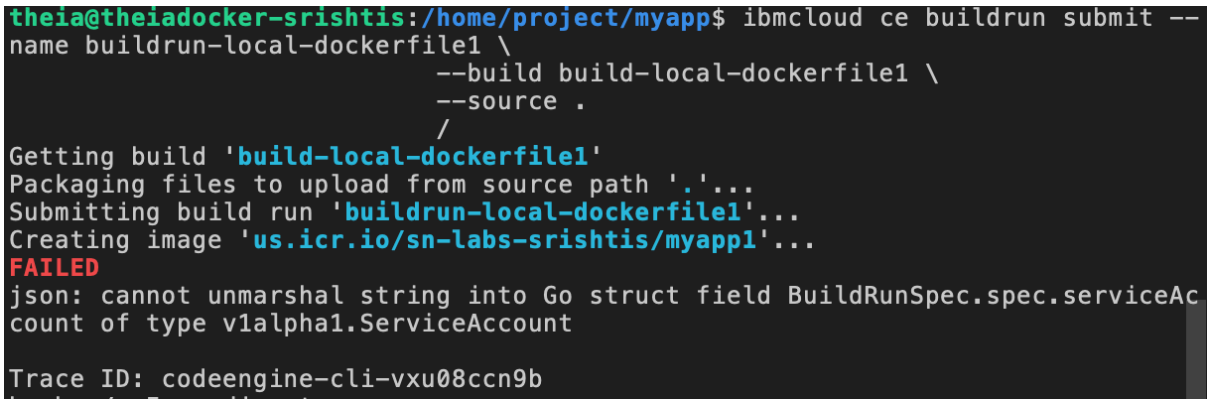
Tomará aproximadamente de 3 a 5 minutos para que todos los pasos en un buildrun terminen de ejecutarse.

Después de ejecutar el comando, debería tomar aproximadamente de 3 a 5 minutos para que veas el siguiente mensaje en la CLI:



buildrun

Nota: En caso de que encuentres el siguiente problema después de ejecutar el comando anterior, aún puedes continuar con los siguientes pasos e ignorar el mensaje de error.



Buildrun

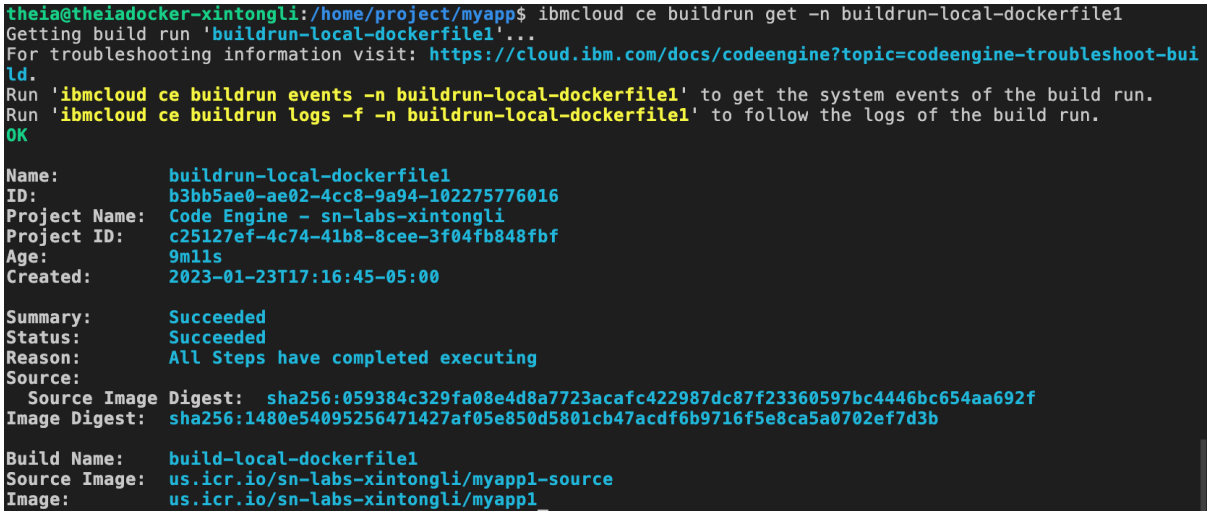
The content of this lab is licensed under Apache 2.0

Para monitorear el progreso del buildrun, utiliza el siguiente comando:

```bash
ibmcloud ce buildrun get -n buildrun-local-dockerfile1
```

Run

Una vez que veas el estado mostrando **Succeeded** (igual que en la siguiente captura de pantalla), eso significa que tu imagen de contenedor se ha creado exitosamente y se ha enviado al registro bajo tu espacio de nombres.



Buildrun Succeeded

Ahora que la imagen del contenedor está lista, necesitas extraer la imagen del Registro de Contenedores y desplegar una aplicación en contenedor utilizando la imagen.

The content of this lab is licensed under Apache 2.0

Desplegando una aplicación contenerizada usando Code Engine

En el paso anterior, creaste y subiste la imagen de la aplicación a tu espacio de nombres en el Container Registry. Vamos a desplegar la aplicación haciendo referencia a la imagen `us.icr.io/${SN_ICR_NAMESPACE}/myapp1` en el Container Registry.

Creando tu aplicación

Code Engine te permite desplegar una aplicación que utiliza una imagen almacenada en IBM Cloud Container Registry con el comando `ibmcloud app create`.

Ejecuta el siguiente comando en tu CLI de Code Engine para desplegar la aplicación:

```bash
ibmcloud ce application create --name demo1 \
                            --image us.icr.io/${SN_ICR_NAMESPACE}/myapp1
                            --registry-secret icr-secret --es 2G \
                            --port 7860 --minscale 1
```

Run

Después de ejecutar el comando, debería tomar aproximadamente de 3 a 5 minutos para que veas el siguiente mensaje en la CLI:

```
theia@theiadocker-xintongli:/home/project$ ibmcloud ce application create --name demo1 --image us.icr.io/sn-labs-xint
ongli/myapp1  --registry-secret icr-secret --es 2G --port 7860 --minscale 1
Creating application 'demo1'...
Configuration 'demo1' is waiting for a Revision to become ready.
Ingress has not yet been reconciled.
Waiting for load balancer to be ready.
Run 'ibmcloud ce application get -n demo1' to check the application status.
OK
```

DeployedApp

Hay algunos argumentos importantes especificados en el comando:

- La aplicación desplegada se llamará `demo1`

The content of this lab is licensed under Apache 2.0

- Hace referencia (utiliza) la imagen `us.icr.io/sn-labs-xintongli/myapp1`.
- Utiliza `2G` de almacenamiento efímero en el contenedor. Para aplicaciones más pequeñas, un valor predeterminado para el almacenamiento efímero, es decir, `--es 400MB`, podría ser suficiente. Necesitas 2G debido al tamaño del modelo GPT2.
- Debes especificar el número de puerto `7860` para la aplicación para que las conexiones y solicitudes de la red pública puedan ser dirigidas a la aplicación cuando lleguen al servidor.
- Configuras `--minscale 1` para asegurarte de que tu aplicación siga funcionando incluso si no hay solicitudes continuas. Esto es importante porque, de lo contrario, tendrías que esperar a que la aplicación comience a ejecutarse cada vez que accedas a su URL.

Accediendo a tu aplicación

¡Esto significa que tu aplicación ha sido desplegada y ahora puedes acceder a ella! Para obtener la URL de tu aplicación, ejecuta `ibmcloud ce app get --name demo1 --output url`. Haz clic en la URL devuelta y deberías poder ver tu aplicación ejecutándose en tu navegador.

También puedes crear un dominio personalizado y asignarlo a tu aplicación. Para obtener información sobre cómo desplegar una aplicación con un dominio personalizado a través de Cloudflare, consulta la Configuración de un Dominio Personalizado para Tu Aplicación de Code Engine.

The content of this lab is licensed under Apache 2.0

Conclusión

¡Felicitaciones por alcanzar este hito!

En conclusión, has navegado exitosamente a través del intrincado proceso de desplegar una aplicación en IBM Cloud utilizando Gradio e IBM Code Engine. Este viaje comenzó con la comprensión de las imágenes de contenedor y los contenedores, esenciales para aislar y ejecutar aplicaciones de manera independiente en diversos entornos. Luego te embarcaste en la tarea práctica de crear una imagen de contenedor, que implicó la recopilación de archivos clave como `demo.py`, `requirements.txt` y `Dockerfile`.

Probar estos componentes localmente aseguró el funcionamiento fluido de tu aplicación antes de desplegarla a través de Docker. Posteriormente, te adentraste en los reinos de IBM Code Engine, una plataforma robusta que no solo aloja tus cargas de trabajo en contenedores, sino que también simplifica el proceso de despliegue al manejar las complejidades de la infraestructura subyacente.

Los pasos finales de tu viaje involucraron aprovechar los recursos de IBM Cloud para construir y enviar tu imagen de contenedor a un registro, seguido del despliegue de tu aplicación y su accesibilidad a través de una URL pública. Este proceso integral, desde las pruebas locales hasta el despliegue en la nube, resalta la integración fluida de Gradio, Docker e IBM Code Engine, culminando en un exitoso despliegue de la aplicación en IBM Cloud.

Las habilidades y conocimientos que has adquirido a través de este proceso son invaluables. Has dominado el arte de desplegar aplicaciones en un entorno de nube, una competencia vital en el mundo tecnológico actual. Celebra tu logro y mira hacia adelante para aprovechar estas habilidades en tus futuros proyectos y emprendimientos.

Author

Sina Nazeri

Connect with me on Linkedin.

The content of this lab is licensed under Apache 2.0

© IBM Corporation. All rights reserved.

Despliega tu aplicación con Code Engine



Introducción

Ahora que tienes Gradio para ayudarte a generar una interfaz de usuario para una aplicación, veamos cómo puedes ejecutar aplicaciones en IBM Cloud y acceder a ellas con una URL pública utilizando IBM Code Engine.

Objetivos de aprendizaje

Al final de este proyecto, podrás:

- Crear una imagen de contenedor
- Crear los archivos necesarios para desplegar tu aplicación en un contenedor
- Crear tu proyecto en Code Engine
- Construir una imagen de contenedor con Code Engine
- Desplegar una aplicación en contenedor usando Code Engine

Imágenes de contenedor y contenedores

Code Engine te permite ejecutar tus aplicaciones en contenedores en IBM Cloud. Un **contenedor** es un entorno o lugar aislado donde una aplicación puede ejecutarse de manera independiente. Los contenedores pueden ejecutarse en

The content of this lab is licensed under Apache 2.0

desarrolladores, servidores físicos, etc. Esto permite que la aplicación en contenedor se ejecute en cualquier lugar y el mecanismo de aislamiento asegura que la aplicación en ejecución no interfiera con el resto del sistema.

Los contenedores se crean a partir de imágenes de contenedor. Una imagen de contenedor es básicamente una instantánea o un plano que indica qué habrá en un contenedor cuando se ejecute. Por lo tanto, para desplegar una aplicación en contenedor, primero necesitas crear la imagen de contenedor de la aplicación.

Creando la imagen de contenedor

Los archivos necesarios para desplegar tu aplicación en un contenedor son los siguientes:

- Puedes usar el marco de Gradio para generar la interfaz de usuario de la aplicación, por ejemplo, el script de Python que contiene el código para crear y lanzar la **gradio.Interface** puede llamarse `demo.py`.
- El código fuente de la aplicación tiene sus dependencias, como bibliotecas que utiliza el código. Por lo tanto, necesitas un archivo `requirements.txt` que especifique todas las bibliotecas de las que depende el código fuente.
- Necesitas un archivo que muestre al entorno de ejecución del contenedor los pasos para ensamblar la imagen del contenedor, llamado `Dockerfile`.

Vamos a crear los tres archivos. Abre una terminal y, mientras estés en el directorio `home/project`, crea un nuevo directorio `myapp` para almacenar los archivos y entra en el directorio con el siguiente comando:

```bash
mkdir myapp
cd myapp
```

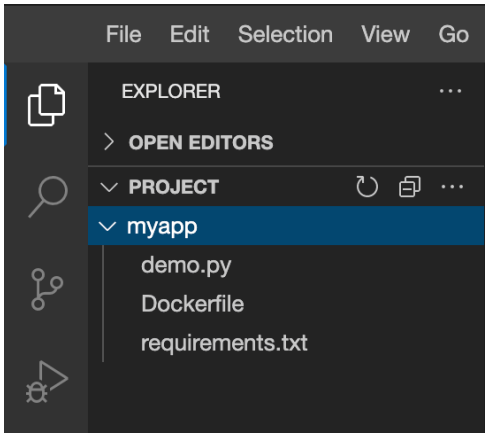
Run

Ahora que estás en el directorio `myapp`, ejecuta el siguiente comando para crear los archivos:

```bash
touch demo.py Dockerfile requirements.txt
```

The content of this lab is licensed under Apache 2.0

Si abres el explorador de archivos, verás los archivos que creaste.



Directorio myapp

A continuación, echemos un vistazo más de cerca a lo que debería incluirse en cada uno de los tres archivos.

Paso 1: Creando require… →

The content of this lab is licensed under Apache 2.0